Adaptive Lease-Based Causal Pruning in Distributed Key-Value Stores

Eric Nelson
Department of Computer Science
The University of Texas at Austin
Austin, Texas
enelson@utexas.edu

Pallavi Desai
Department of Computer Science
The University of Texas at Austin
Austin, Texas
pallavi.desai@utexas.edu

***Abstract*—Causality tracking in high-churn distributed systems often leads to metadata bloat, known as the actor explosion problem. This paper evaluates the trade-offs between causal safety and active pruning through a lease-based mechanism for Dotted Version Vectors (DVV).**

***Index Terms*—distributed systems, causal consistency, version vectors, churn, metadata pruning**

I. Introduction

Tracking causality is fundamental to maintaining consistency in leaderless distributed systems. However, as membership churn increases, the overhead of maintaining logical clocks becomes a primary bottleneck...

II. Related Works

A. Foundational Causality and the Scaling Bottleneck

The tracking of causality in distributed systems originated with Lamport's scalar clocks, which provided a total ordering of events but lacked the ability to characterize concurrent execution [1]. This limitation was resolved by the introduction of Vector Clocks (VCs) by Fidge and Mattern, which accurately capture the "happened-before" relationship by maintaining an array of logical counters, one for each participating process [2], [3]. However, traditional VCs rely on the assumption of static system membership. In environments characterized by churn, the metadata overhead of VCs grows linearly, bounded by $O(N)$ where N represents the lifetime sum of all unique processes. This results in the "actor explosion" problem, rendering pure VCs unsuitable for long-running, dynamic architectures where processes frequently join and leave.

B. Logical Clocks for Dynamic Membership

To address the $O(N)$ scaling constraint, several mechanisms have been proposed to adapt vector-like structures to dynamic environments. Landes introduced Dynamic Vector Clocks, formalizing the addition and retirement of entries using distributed consensus, though the coordination overhead proved prohibitive for highly partitioned systems [4]. Torres-Rojas and Ahamad proposed Plausible Clocks, which guarantee a constant-size metadata overhead by mapping an unbounded number of processes to a fixed-size vector [5]. While highly space-efficient, Plausible Clocks trade accuracy for size, leading to false positives in conflict detection when distinct processes map to the same vector index.

A mathematically rigorous approach to extreme churn was introduced by Almeida *et al.* with Interval Tree Clocks (ITC) [6]. ITC eliminates the need for global identifiers entirely, allowing nodes to dynamically "fork" and "join" identity spaces. While ITC successfully bounds metadata growth during optimal operation, it suffers from severe identifier fragmentation under asymmetric churn patterns, leading to bit-string growth that can eventually degrade performance.

C. Version Vectors and Optimistic Replication

In the context of distributed data stores, vector clocks are typically adapted into Version Vectors (VVs) to manage optimistic replication and conflict resolution, as originally implemented in the LOCUS distributed operating system [7]. Modern distributed architectures, heavily influenced by Dynamo [8], initially relied on traditional VVs but quickly encountered severe metadata bloat and "sibling explosion" when managing transient clients or ephemeral storage nodes.

Preguiça *et al.* directly addressed this in highly available key-value stores with the introduction of Dotted Version Vectors (DVVs) [9]. By decoupling the most recent event (the "dot") from the summarized causal context, DVVs successfully constrain metadata growth to the degree of data replication rather than the number of active clients. DVVs represent the current industry standard for avoiding false conflicts in leaderless systems. However, the original formalization of DVVs relies on logical subsumption for metadata compaction; it assumes that retired nodes will eventually synchronize or that an external mechanism will handle the removal of defunct entries.

D. Metadata Pruning and Garbage Collection Strategies

While structural innovations like DVVs compress causal history, systems experiencing continuous, long-term churn must eventually rely on active pruning to prevent memory exhaustion and network saturation. In practice, industry systems rely on heuristics rather than formally verified protocols. For instance, Apache Cassandra utilizes a passive time-to-live (TTL) expiration mechanism to aggressively delete tombstones and causal metadata [10]. Similarly, systems like Riak pair

DVVs with active anti-entropy background services using Merkle Trees to periodically reconcile state [11].

These implementations highlight a critical gap in the academic literature: the tension between causal safety and active pruning. While extensive research exists on ensuring causal correctness under the assumption of persistent metadata, there is limited formal evaluation of expiration-driven pruning mechanisms. Current literature treats metadata expiration either as a static, blunt-force TTL or relies on expensive fallback state transfers. Our work formalizes this expiration not as a static threshold, but as an adaptive, cooperative lease, providing a framework to bound causal metadata while minimizing the frequency and impact of expensive anti-entropy operations.

REFERENCES

[1] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978.

[2] C. J. Fidge, "Timestamps in message-passing systems that preserve the partial ordering," in *Proc. 11th Australian Computer Science Conference (ACSC)*, 1988, pp. 56–66.

[3] F. Mattern, "Virtual time and global states of distributed systems," in *Parallel and Distributed Algorithms*, M. Cosnard *et al.*, Eds. Amsterdam, The Netherlands: North-Holland, 1989, pp. 215–226.

[4] C. Landes, "Dynamic vector clocks for consistent ordering of events in dynamic distributed applications," in *Proc. IFIP/IEEE Int. Conf. on Distributed Platforms*, 1996, pp. 31–43.

[5] R. S. Torres-Rojas and M. Ahamad, "Plausible clocks: constant size logical clocks for distributed systems," *Distributed Computing*, vol. 12, no. 4, pp. 179–195, Oct. 1999.

[6] P. S. Almeida, C. Baquero, and V. Fonte, "Interval tree clocks: A logical clock for dynamic systems," in *Proc. 12th Int. Conf. Principles of Distributed Systems (OPODIS)*, Luxor, Egypt, 2008, pp. 259–274.

[7] D. S. Parker *et al.*, "Detection of mutual inconsistency in distributed systems," *IEEE Transactions on Software Engineering*, vol. SE-9, no. 3, pp. 240–247, May 1983.

[8] G. DeCandia *et al.*, "Dynamo: amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. on Operating Systems Principles (SOSP)*, Stevenson, WA, USA, 2007, pp. 205–220.

[9] N. Preguiça, C. Baquero, P. S. Almeida, V. Fonte, and R. Gonçalves, "Dotted version vectors: Logical clocks for optimistic replication," in *Proc. Int. Conf. on Networked Systems (NETYS)*, Marrakech, Morocco, 2012, pp. 1–15.

[10] A. Lakshman and P. Malik, "Cassandra: a decentralized structured storage system," *Operating Systems Review*, vol. 44, no. 2, pp. 35–40, Apr. 2010.

[11] Basho Technologies, "Active Anti-Entropy," Riak KV Documentation. Accessed: May 4, 2026. [Online]. Available: https://docs.riak.com/riak/kv/latest/learn/concepts/active-anti-entropy/index.html